

SAPIENZA
UNIVERSITÀ DI ROMA

Implementation of Bine Tree Algorithms in NCCL for Multi-GPU Systems

Facoltà di Ingegneria dell'Informazione,
Informatica e Statistica
Laurea in Informatica

Daniele Savino
ID number 2140356

Advisor
Prof. Daniele De Sensi

Academic Year 2026/2027

Thesis not yet defended

Implementation of Bine Tree Algorithmsin NCCL for Multi-GPU Systems
Bachelor thesis. Sapienza University of Rome

© 2027 Daniele Savino. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Website: https://github.com/DanieleSavino
Author's email: savino.2140356@studenti.uniroma1.it

Abstract

[TODO: Abstract]

Acknowledgments

MeluXina. *The experiments were performed on the Luxembourg national supercomputer MeluXina. The author gratefully acknowledge the LuxProvide teams for their expert support.*





Contents

Acronyms **viii**

1 Introduction **1**

2 Background and Related Work **2**
2.1 Collective communication 2
2.2 Ring, tree and butterfly-style algorithms 3
2.2.1 Ring 3
2.2.2 Binomial tree 4
2.2.3 Ring versus tree: $\mathcal{O}(p)$ versus $\mathcal{O}(\log p)$ 6
2.2.4 Butterfly 7
2.2.5 Bine trees 7
2.2.6 Bine butterflies 11

3 NCCL **12**
3.1 Overview 12
3.2 Topology and graphs 13
3.3 Channels 13
3.4 Transports 14
3.5 Proxy 15
3.6 Protocols 15
3.7 Algorithms 16
3.8 Work distribution across channels 16
3.8.1 Traffic-based splitting 16
3.8.2 Continuous byte distribution (CBD) 16
3.8.3 Chunking 18
3.9 Device side 18
3.9.1 Kernel structure 18
3.9.2 Primitives 18
3.9.3 Writing a new kernel 19

4 Bine Tree Algorithms **20**
4.1 Schedule construction 20
4.1.1 Output and notation 20
4.1.2 Definition of the schedule 21
4.1.3 Example: $p = 8$ 21
4.1.4 Computing the tables 21

4.1.5 Arbitrary root . . . 22
4.2 Broadcast and reduce . . . 23
4.2.1 Integration in NCCL . . . 23
4.2.2 Kernels . . . 23
4.3 Testing . . . 25
4.3.1 Invariants . . . 26
4.3.2 Why they hold and what they guarantee . . . 26
4.3.3 Scope and limits . . . 27
4.4 Experimental results . . . 28
4.4.1 `SIMPLE` protocol . . . 29
4.4.2 `LL` protocol . . . 29
4.4.3 Summary . . . 32

5 Butterfly-style Algorithms **34**

Bibliography **35**

List of Figures

2.1 Communication pattern of a ring over $p = 8$ ranks. At every step, rank r sends to rank $(r + 1) \bmod p$. 3
2.2 Distance-halving binomial tree over $p = 8$ ranks, rooted at rank 0. Each edge is labeled with the step i at which it is used in a broadcast. 5
2.3 Distance-halving binomial tree and Bine tree over $p = 8$ ranks, rooted at rank 0, with the ranks placed on a circle. Solid, dashed and dotted arrows are the edges used at steps 0, 1 and 2. The distances between the communicating ranks are $4, 2, 1$ for the binomial tree and $3, 1, 1$ for the Bine tree. 9

4.1 Broadcast: mean time versus number of ranks, Bine (blue) against ring (red), for six message sizes. Solid lines with filled markers: `SIMPLE` protocol; dashed lines with hollow markers: `LL` protocol. 28
4.2 Reduce: mean time versus number of ranks, Bine (blue) against ring (red), for six message sizes. Solid lines with filled markers: `SIMPLE` protocol; dashed lines with hollow markers: `LL` protocol. 29
4.3 Ring allgather: mean time versus number of ranks, `SIMPLE` (solid, filled markers) against `LL` (dashed, hollow markers), for six block sizes, up to $p = 32$. 31

List of Tables

2.1 Notation used in the thesis. 3
2.2 Cost of the ring and of the binomial tree algorithms ($p = 2^s$). The ring broadcast and reduce are pipelined with k chunks (the reduce adds $\frac{n}{k}\gamma$ to each step). A dash means that the algorithm is not used for that collective. 6
2.3 Ring versus binomial tree in the model of Sec. 2.1 ($p = 2^s$, ring with the optimal number of chunks). Small n: time of the ring over time of the tree, $(p-1)/s$. Large n: time of the tree over time of the ring, s. $x^* = n^*\beta/\alpha$ is the crossover vector size in units of α/β. 7
2.4 Negabinary representation $\overline{r} = \mathit{rank2nb}(r)$, run $u(r)$ and receive step i_r for $p = 8$ ($s = 3$). 9
2.5 Distance between the ranks communicating at a step with $k = s-i$ in a distance-halving binomial tree (2^{k-1}) and in a Bine tree ($|(1-(-2)^k)/3|$). 10

4.1 NCCL operations used by the Bine broadcast and reduce. The direct operation is used when the zero-copy path is available. 26
4.2 Largest measured message size for which `LL` is faster than `SIMPLE`; the crossover lies between this size and the next measured one (1 kB, 16 kB, 256 kB, 1 MB, 4 MB, 64 MB). The Bine and ring entries are the same for broadcast and reduce; the allgather (ring only, Fig. 4.3) was measured up to $p = 32$, a dash means not measured. 30
4.3 Best protocol of each algorithm and speedup of Bine over the ring, both with their best protocol, at $p = 128$. A value below 1 means that the ring is faster. 32

Acronyms

Chapter 1

Introduction

[TODO: Introduction]

Chapter 2

Background and Related Work

This chapter introduces collective operations and the algorithms that the rest of the thesis builds on. It describes the ring and the standard binomial tree, which are the baselines of the thesis, and the Bine tree [2], which is the algorithm implemented in NCCL in the following chapters. The notation introduced here is used throughout the thesis.

2.1 Collective communication

A *collective operation* is executed by all the p processes (*ranks*) of a communicator, each identified by a rank identifier $r \in [0, p-1]$. The operations considered in this thesis are the following.

Broadcast. The root t holds a vector of n bytes, and every rank must end up with a copy of it.

Reduce. Every rank holds a vector of n bytes, and the root t must obtain their element-wise combination (e.g., the sum).

Scatter and gather. In a scatter, the root holds p blocks and rank r must receive the r-th one. The gather is the opposite operation: the root collects one block from every rank.

Allgather. Every rank holds one block, and every rank must obtain the concatenation of all the blocks.

Reduce-scatter. Every rank holds a vector of n bytes divided in p blocks, and rank r must obtain the r-th block of the element-wise combination of all the vectors.

Allreduce. Like the reduce, but every rank obtains the result.

The combination operator is assumed to be associative and commutative, as it is for the operators supported by NCCL (sum, product, maximum, minimum, average). Several collectives are related: the allreduce can be obtained as a reduce-scatter followed by an allgather, and the broadcast as a scatter followed by an allgather.

Table 2.1. Notation used in the thesis.

Symbol	Definition
p	Number of ranks.
r	Rank identifier, $r \in [0, p-1]$.
t	Rank identifier of the root of the collective.
n	Size of the vector of the collective, in bytes.
s	Number of steps of a tree algorithm, $s = \log_2 p$ (p a power of two).
α, β, γ	Latency, time per byte transferred, time per byte reduced.
$\oplus$	Bitwise exclusive or (XOR).
$rank2nb(r)$	Negabinary representation of the rank r (Sec. 2.2.5).
$nb2rank(x)$	Rank identifier whose negabinary representation is x.

Cost model. To compare algorithms we use the usual latency–bandwidth model [3, 7]. Sending a message of m bytes between two ranks takes $\alpha + m\beta$, where α is the latency and β the time to transfer one byte, and combining m bytes locally takes $m\gamma$. Algorithms proceed in *steps*, and at each step a rank sends at most one message and receives at most one. The model ignores the topology of the network: all the pairs of ranks are assumed to be at the same distance. This is precisely the assumption that the Bine tree relaxes (Sec. 2.2.5). Table 2.1 summarizes the notation.

2.2 Ring, tree and butterfly-style algorithms

Most collective algorithms are built from a small number of communication patterns. This section describes the three families relevant for this thesis, the ring, the binomial tree and the butterfly, and then the Bine variants of the tree and of the butterfly. Throughout, $p = 2^s$ unless stated otherwise.

2.2.1 Ring

In a ring algorithm the ranks are arranged in a logical ring, and at every step rank r sends to rank $(r+1) \bmod p$ and receives from rank $(r-1) \bmod p$ (Fig. 2.1). Each rank talks to only two peers, and all the ranks send and receive at every step.

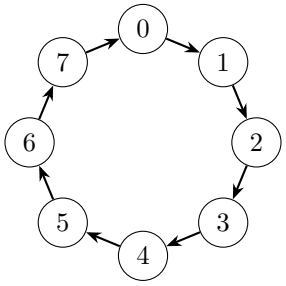


Figure 2.1. Communication pattern of a ring over $p = 8$ ranks. At every step, rank r sends to rank $(r+1) \bmod p$.

Allgather, reduce-scatter and allreduce. In the ring allgather, the vector is divided in p blocks of n/p bytes. At the first step each rank sends its own block, and

at each following step it forwards the block received at the previous one, so that after $p-1$ steps every rank has all the blocks. The reduce-scatter is the same pattern in which the block received at each step is combined with the local one before being forwarded. The allreduce is a reduce-scatter followed by an allgather. The costs are

$$T_{\text{allgather}} = (p-1)\,\alpha + \tfrac{p-1}{p}\,n\beta, \tag{2.1}$$

$$T_{\text{reduce-scatter}} = (p-1)\,\alpha + \tfrac{p-1}{p}\,n\beta + \tfrac{p-1}{p}\,n\gamma, \tag{2.2}$$

$$T_{\text{allreduce}} = 2(p-1)\,\alpha + 2\tfrac{p-1}{p}\,n\beta + \tfrac{p-1}{p}\,n\gamma. \tag{2.3}$$

The bandwidth term is optimal: every rank sends and receives a number of bytes that does not grow with p and that matches the lower bound for these collectives.

Broadcast and reduce. The ring can also be used for the broadcast and the reduce by pipelining. The vector is split in k chunks that travel along the ring starting from the root: after the first chunk has reached the last rank, a new chunk is delivered at every step. The broadcast takes $p+k-2$ steps of n/k bytes each,

$$T_{\text{broadcast}} = (p+k-2)\left(\alpha + \tfrac{n}{k}\beta\right), \tag{2.4}$$

and the reduce is the same pattern in the opposite direction, with an additional $\frac{n}{k}\gamma$ per step. For a large k the bandwidth term tends to $n\beta$, independently of p, but the latency term grows linearly with the number of ranks, and chunks cannot be made arbitrarily small without paying α for each of them.

Locality. Since each rank communicates only with its two neighbours, a ring whose ranks are mapped consecutively on the machine uses very few long-distance links. For example, on a network organized in groups, only two of the p messages sent at each step cross a group boundary for every boundary [2]. This is why the ring is competitive for large vectors even on oversubscribed networks. Its weakness is the number of steps: the performance is suboptimal for large numbers of ranks and small to medium vectors [2], where the latency term $\mathcal{O}(p)\ \alpha$ dominates.

2.2.2 Binomial tree

Here, *tree* always means the standard binomial tree, not the double binary tree used by NCCL (Sec. 3.2).

Construction. A binomial tree of order 0 is a single node. A binomial tree of order k is obtained by linking two binomial trees of order $k-1$, making the root of one of them a child of the root of the other (Fig. 2.2). A tree of order s has 2^s nodes, depth s and 2^s-1 edges; the root has s children, and the number of nodes at depth d is the binomial coefficient $\binom{s}{d}$, which gives the tree its name. In a collective, each node of the tree is a rank and each edge is a communication between two ranks. The assignment of ranks to nodes is arbitrary, and any assignment gives a correct algorithm. For the collectives that include a computation (e.g., the reduce) this requires that the operator is associative and, since the assignment of ranks to nodes is arbitrary, commutative.

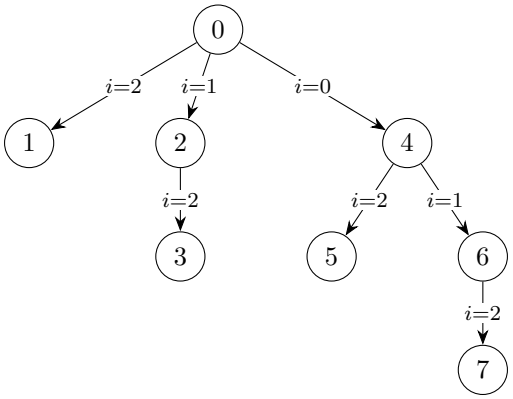


Figure 2.2. Distance-halving binomial tree over $p = 8$ ranks, rooted at rank 0. Each edge is labeled with the step i at which it is used in a broadcast.

Broadcast. The root first sends the vector to one rank. Then, at each step, every rank that already holds the data sends it to a rank that does not, so that the number of ranks holding the data doubles at each step and the broadcast completes in $s = \log_2 p$ steps. The algorithm is correct for any choice of the destinations, as long as a rank never sends to a rank that already has the data, but different choices give different trees. The two standard ones differ in the distance between the ranks that communicate, where the distance between r and q is $|r - q|$. Let $r' = (r - t) \bmod p$ be the rank relative to the root t.

- In the *distance-doubling* tree, used for example by Open MPI [2], at step i rank r' sends to $r' \oplus 2^i$, so the distance doubles at every step: $1, 2, 4, \ldots, p/2$. A rank $r' \neq 0$ receives at the step given by the position of its most significant set bit.

- In the *distance-halving* tree, used for example by MPICH [2], at step i rank r' sends to $r' \oplus 2^{s-i-1}$, so the distance halves at every step: $p/2, \ldots, 2, 1$. A rank $r' \neq 0$ receives at the step $s - 1 - \nu$, where ν is the position of its least significant set bit.

Figure 2.2 shows the distance-halving tree for $p = 8$ and $t = 0$. Rank 0 sends to 4, 2, 1 at steps 0, 1, 2; rank 4 sends to 6 and 5 at steps 1 and 2; rank 2 sends to 3 at step 2; and rank 6 sends to 7 at step 2. The distance-halving tree is the one the thesis compares against, because it is the one that the Bine tree modifies.

Other collectives. The reduce uses the same tree with the edges reversed: the leaves send first, and each rank combines the vectors received from its children with its own before sending to its parent. In the scatter the root sends to its child the part of the vector (the blocks) destined to the subtree of that child, and the same is repeated at each level; the gather is the opposite operation. In both, the amount of data per message halves (or doubles) at each step, so that the total volume handled by the root is $\frac{p-1}{p}n$. For a large vector, the broadcast can also be implemented as a scatter followed by an allgather [7].

Table 2.2. Cost of the ring and of the binomial tree algorithms ($p = 2^s$). The ring broadcast and reduce are pipelined with k chunks (the reduce adds $\frac{n}{k}\gamma$ to each step). A dash means that the algorithm is not used for that collective.

Collective	Ring	Binomial tree
Broadcast	$(p+k-2)(\alpha+\frac{n}{k}\beta)$	$s\,(\alpha+n\beta)$
Reduce	$(p+k-2)(\alpha+\frac{n}{k}\beta+\frac{n}{k}\gamma)$	$s\,(\alpha+n\beta+n\gamma)$
Scatter, gather	–	$s\,\alpha+\frac{p-1}{p}n\beta$
Allgather	$(p-1)\alpha+\frac{p-1}{p}n\beta$	–
Reduce-scatter	$(p-1)\alpha+\frac{p-1}{p}n\beta+\frac{p-1}{p}n\gamma$	–
Allreduce	$2(p-1)\alpha+2\frac{p-1}{p}n\beta+\frac{p-1}{p}n\gamma$	–

Cost. In the broadcast and in the reduce, every edge carries the whole vector, so

$$T_{\text{broadcast}} = s\,(\alpha + n\beta), \qquad T_{\text{reduce}} = s\,(\alpha + n\beta + n\gamma), \tag{2.5}$$

while the scatter and the gather take $s\,\alpha + \frac{p-1}{p}n\beta$. Table 2.2 compares them with the ring. The tree has a logarithmic latency term, which makes it the better choice for small and medium vectors and large p. Its bandwidth term is $s\,n\beta$, i.e., it grows with p, while the one of the pipelined ring does not, so for large vectors the ring wins. The same trade-off appears in the measurements of Chapter 4.

Number of ranks not a power of two. Binomial trees can be adapted to any p by running the algorithm on the $p' = 2^{\lfloor \log_2 p \rfloor}$ ranks and by adding a step in which the remaining ranks exchange data with ranks of the first group [2].

Communication locality. The cost model of Table 2.2 assumes that all pairs of ranks are equally distant, but on a real machine this is not true. The two trees above have the same number of steps and the same total volume of data, but they can generate a very different traffic on the slow links [2]. Consider a broadcast of n bytes over eight nodes connected through a 2:1 oversubscribed fat tree, where each leaf switch serves two consecutive ranks, i.e., $\{0,1\}, \{2,3\}, \{4,5\}, \{6,7\}$, and the uplinks are the bottleneck. In the distance-doubling tree, the transfers at the second and third steps ($0 \to 2$ and $1 \to 3$; then $0 \to 4$, $1 \to 5$, $2 \to 6$, $3 \to 7$) all cross the uplinks, for a total of $2n + 4n = 6n$ bytes, and the two transfers of the second step even share the uplink of the first switch. In the distance-halving tree, only $0 \to 4$ (n bytes) and $0 \to 2$, $4 \to 6$ ($2n$ bytes) cross the uplinks, for $3n$ bytes in total, and the last step, which is the one with the most transfers, is entirely local. The intuition is that closer communication should be preferred when more ranks are sending. This example also assumes that rank identifiers are close to the physical position of the ranks, which holds when the ranks are mapped consecutively on the machine (e.g., under the default block distribution of Slurm).

2.2.3 Ring versus tree: $\mathcal{O}(p)$ versus $\mathcal{O}(\log p)$

For the broadcast and the reduce, the two baselines of the thesis differ in the term that grows with the number of ranks (Table 2.2). The pipelined ring has a bandwidth

Table 2.3. Ring versus binomial tree in the model of Sec. 2.1 ($p = 2^s$, ring with the optimal number of chunks). Small n: time of the ring over time of the tree, $(p-1)/s$. Large n: time of the tree over time of the ring, s. $x^* = n^*\beta/\alpha$ is the crossover vector size in units of α/β.

p	4	8	16	32	64	128
Small n: ring/tree	1.5×	2.3×	3.8×	6.2×	10.5×	18.1×
Large n: tree/ring	2×	3×	4×	5×	6×	7×
Crossover x^*	8.0	8.7	12.0	17.8	27.8	44.9

term that does not depend on p but a latency term linear in p, since the first chunk needs $p-1$ steps to reach the last rank. The tree has a latency term logarithmic in p, but every edge carries the whole vector, so its bandwidth term grows as $s\,n\beta$.

The ring cost of Eq. (2.4) depends on the number of chunks k. Minimizing it over k gives $k^* = \sqrt{(p-2)\,n\beta/\alpha}$ and

$$T^*_{\text{ring}} = \left(\sqrt{(p-2)\,\alpha} + \sqrt{n\beta}\right)^2 = (p-2)\,\alpha + n\beta + 2\sqrt{(p-2)\,\alpha\,n\beta}, \qquad (2.6)$$

valid when $k^* \geq 1$; otherwise the best choice is $k = 1$ and the ring takes $(p-1)(\alpha + n\beta)$ [1]. The same square-root form is obtained for the pipelined broadcast of other topologies, e.g., $\left(\sqrt{(\lceil \log_2 p \rceil - 1)\alpha} + \sqrt{n\beta}\right)^2$ for the optimal algorithm of [8]. This is to be compared with $T_{\text{tree}} = s(\alpha + n\beta)$ of Eq. (2.5). For the reduce, the same holds with β replaced by $\beta + \gamma$. Two regimes follow.

- *Small vectors* ($n\beta \ll \alpha$). The ring is latency-bound and costs about $(p-1)\alpha$, against $s\,\alpha$ for the tree: the tree is faster by a factor $(p-1)/\log_2 p$, that is $\mathcal{O}(p)$ versus $\mathcal{O}(\log p)$ steps.
- *Large vectors* ($n\beta \gg (p-2)\alpha$). The ring tends to $n\beta$, independently of p, while the tree costs $s\,n\beta$: the ring is faster by a factor $\log_2 p$.

The crossover is at $n^* = x^*\,\alpha/\beta$, where x^* solves $s(1+x) = (p-2) + x + 2\sqrt{(p-2)x}$ and α/β is the bandwidth-delay product of a link. Table 2.3 gives x^* and the two factors. The crossover moves to larger vectors as p grows, so the larger the job, the larger the range of sizes in which the tree is preferable.

The model assumes that the number of chunks is free and that all links are equal. In NCCL the chunk is bounded by the size of a slot (Sec. 3.8.3), so a small message is a single chunk and the ring behaves as in the $k = 1$ case. The model also ignores the network topology, which is where the Bine tree differs from the binomial one: the two have the same cost in this model (Sec. 2.2.5), so the comparison above holds for both, and Bine only changes how much of the traffic crosses the slow links.

2.2.4 Butterfly

2.2.5 Bine trees

Bine (*binomial negabinary*) trees are a family of binomial-tree algorithms that reduce the distance between the ranks that communicate, and so the amount of data

that crosses the slow links of the network [2]. They keep the generality of binomial trees: they have the same number of steps, the same number of messages and the same communication cost in the model of Sec. 2.1, they do not need to know the topology or the way the ranks are allocated, and their schedule is computed from the rank identifiers alone, with the same simplicity. In this thesis we only use the distance-halving Bine tree, which we describe in this section; the distance-doubling variant belongs to the butterfly algorithms (Sec. 2.2.6).

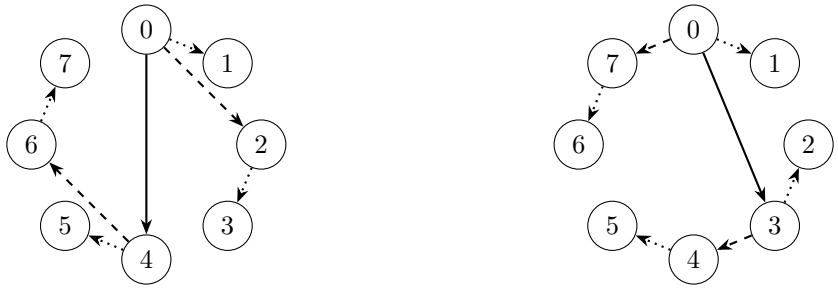
Modular distance. A standard binomial tree does not use the physical distance between nodes, which would tie the algorithm to a specific machine, but the difference between the rank identifiers. This approximates the physical distance when the ranks are mapped consecutively on the machine. If this is not the case, the ranks can be remapped to a block distribution when the communicator is created, e.g., by sorting them by hostname when hostnames are numbered consecutively across groups [2]. Bine trees use a similar approach, but with the *modular distance*: the p ranks are arranged on a circle $0, 1, \ldots, p-1$, and the distance between two ranks is the length of the shortest path on the circle,

$$d(r, q) = \min((r - q) \bmod p,\ (q - r) \bmod p). \tag{2.7}$$

The motivation is that, when a job spans several groups, ranks 0, 1 and 2 may well belong to three different groups. The standard distance would consider 0 and 2 farther apart than 0 and 1, but both pairs communicate between groups, so they should be treated as equal, as the modular distance does for ranks 0 and $p-1$ as well. Counterexamples exist in which the standard distance is more accurate, but for jobs spanning multiple groups, which is the common case, the modular distance is the better approximation [2].

Construction. The Bine tree is built recursively, like the binomial tree, but the way the two subtrees are joined is chosen to minimize the modular distance between the two roots. A Bine tree of order 1 (two nodes) is identical to a binomial tree. To build the tree of order 2, an order-1 tree is connected to a copy of itself that is *mirrored* (reflected on the vertical axis) and placed to its *left*, rather than to the right as for a binomial tree. The roots are then at modular distance 1 instead of 2. The same rule gives the order-3 tree from two order-2 trees: placing the mirrored tree on the side that gives the shortest distance between the roots yields a distance of 3, against 4 for the binomial tree. The ranks are numbered from the root from left to right, wrapping around when the rightmost node is reached, so that the ranks to the left of the root are those with the highest identifiers. Figure 2.3 compares the distance-halving binomial tree and the Bine tree for $p = 8$.

Negabinary representation. The schedule is computed with the *negabinary* (base -2) representation of the ranks, in which a number is written as a sum of powers of -2 instead of 2. For example, $2 = 110_{-2}$, since $1 \cdot (-2)^2 + 1 \cdot (-2)^1 + 0 \cdot (-2)^0 = 4 - 2 = 2$. Unlike the binary representation, the negabinary one represents positive and negative integers without a sign (e.g., $011_{-2} = -1$), so on a fixed number of bits not every string is a non-negative rank. With s bits, the largest non-negative number m



(a) Binomial tree. **(b)** Bine tree.

Figure 2.3. Distance-halving binomial tree and Bine tree over $p = 8$ ranks, rooted at rank 0, with the ranks placed on a circle. Solid, dashed and dotted arrows are the edges used at steps 0, 1 and 2. The distances between the communicating ranks are $4, 2, 1$ for the binomial tree and $3, 1, 1$ for the Bine tree.

Table 2.4. Negabinary representation $\overline{r} = \mathit{rank2nb}(r)$, run $u(r)$ and receive step i_r for $p = 8$ ($s = 3$).

r	0	1	2	3	4	5	6	7
$\overline{r}$	000	001	110	111	100	101	010	011
$u(r)$	–	1	1	3	2	1	1	2
i_r	-1	2	2	0	1	2	2	1

that can be represented is obtained by setting to one all the even positions and to zero all the odd ones, since only even powers of -2 are positive; for $s = 6$, $m = 010101_{-2} = 16 + 4 + 1 = 21$. The construction of the Bine tree places the rightmost rank at m. Consequently, the ranks $r \in [0, m]$ are represented by their own negabinary value, while a rank $r > m$ (on the left of the root) is represented by the negabinary value of $r - p$. For $p = 8$ ($s = 3$), $m = 101_{-2} = 5$, and rank 6 is represented by $6 - 8 = -2 = 010_{-2}$. We write $\mathit{rank2nb}(r)$ for the representation of r ($\mathit{rank2nb}(r, p)$ if p needs to be explicit) and $\mathit{nb2rank}(x)$ for the inverse function; both can be computed efficiently with bit masking and one addition or subtraction [2]. Table 2.4 lists the representations for $p = 8$.

Communication partner. Consider a broadcast with root 0. At step $i \in [0, s-1]$ every rank that already holds the data sends it to the rank whose negabinary representation differs from its own in the $s - i$ least significant bits:

$$\pi_i(r) = \mathit{nb2rank}\big(\mathit{rank2nb}(r) \oplus M_i\big), \qquad M_i = 2^{s-i} - 1, \tag{2.8}$$

where M_i is the mask with the $s - i$ least significant bits set to one. The root has the representation $0 \cdots 0$, so at step i it sends to the rank represented by M_i, i.e., $1 \cdots 1$ on $s - i$ bits. For the other ranks, we need to know the step at which they join the tree. Let $u(r)$ be the number of consecutive least significant bits of $\mathit{rank2nb}(r)$ that are equal to the least significant one (e.g., $u = 3$ for 1000, and $u = 2$ for 1011). A rank $r \neq 0$ receives the data from its parent at step

$$i_r = s - u(r), \qquad i_r \in [0, s-1], \tag{2.9}$$

Table 2.5. Distance between the ranks communicating at a step with $k = s - i$ in a distance-halving binomial tree (2^{k-1}) and in a Bine tree ($|(1-(-2)^k)/3|$).

$k = s - i$	1	2	3	4	5	6
Binomial tree	1	2	4	8	16	32
Bine tree	1	1	3	5	11	21
Ratio	1.00	0.50	0.75	0.63	0.69	0.66

and forwards it at every step $i > i_r$ using Eq. (2.8). Its parent is $\pi_{i_r}(r)$. For $p = 8$ (Table 2.4), rank 3 (111) receives at step 0 from $nb2rank(111 \oplus 111) = 0$, and sends at step 1 to $nb2rank(111 \oplus 011) = nb2rank(100) = 4$ and at step 2 to $nb2rank(111 \oplus 001) = nb2rank(110) = 2$. The resulting edges are those of Fig. 2.3. The implementation of this computation in NCCL is described in Sec. 4.1.

Arbitrary root. For a root $t \neq 0$, the schedule is obtained by rotating the one of root 0: each rank identifier r is replaced by $(r - t) \bmod p$. The rotation is a relabeling, so it does not change the structure of the tree or the distances between the communicating ranks (Sec. 4.1.5).

Distance reduction. We now quantify the gain with respect to the standard distance-halving binomial tree. In the binomial tree, the two ranks that communicate at step i differ in the bit in position $s - i - 1$, so their distance is

$$\delta_{\text{binomial}}(i) = 2^{s-i-1}. \tag{2.10}$$

In the Bine tree, their negabinary representations differ in the $s - i$ least significant bits, which are all 1 for one rank and all 0 for the other. Their distance is therefore

$$\delta_{\text{bine}}(i) = \left| \sum_{j=0}^{s-i-1} (-2)^j \right| = \left| \frac{1-(-2)^{s-i}}{3} \right| \approx \frac{2^{s-i}}{3}. \tag{2.11}$$

Since for a binomial tree the maximum distance between two ranks is $p/2$, the distance coincides with the modular distance, and the two can be compared directly:

$$\frac{\delta_{\text{bine}}(i)}{\delta_{\text{binomial}}(i)} \approx \frac{2^{s-i}/3}{2^{s-i-1}} = \frac{2}{3}. \tag{2.12}$$

Asymptotically, communicating ranks are therefore at a modular distance about one third shorter; for the small step sizes of Table 2.5 the reduction ranges between 25% and 50%. The distances do not halve exactly at each step, but they differ by at most one from the ideal halving.

Number of ranks not a power of two. For p even, the construction can be used as is, but some ranks may be reached twice, which would break correctness (e.g., a contribution would be reduced twice). In that case, one of the two subtrees rooted at the rank that is reached twice is removed, namely the one reached later, which is always the smaller one. For p odd the construction cannot be applied directly, since

communications happen in pairs, and the techniques for binomial trees mentioned above must be used [2]. The implementation of this thesis is limited to powers of two (Chapter 4).

2.2.6 Bine butterflies

[TODO: Afer Butterfly implementation (ch. 05)]

Chapter 3

NCCL

This chapter describes the parts of NCCL [6] that the rest of the thesis relies on. It is not a complete description of the library: it follows a collective from the moment it is issued to the moment the data is moved, and introduces, in this order, the topology graph, the channels, the transports, the proxy, the protocols, the way the work is distributed across channels, and the device-side primitives that the kernels are written with. The functions and data structures used by the Bine kernels in the following chapters are defined here and referenced from there. For a more detailed analysis of the library see [4]. The source excerpts and names refer to the NCCL code base the thesis was developed on, and some of them (flag names, internal helpers) can change between versions.

3.1 Overview

NCCL is a library of collective operations (broadcast, reduce, all-reduce, all-gather, reduce-scatter) and point-to-point operations for NVIDIA GPUs. Each GPU is a *rank* of a *communicator*. The library is organised in two halves that cooperate during a collective.

Host side. At communicator creation, NCCL discovers the hardware topology, searches the communication graphs (Sec. 3.2), creates the channels (Sec. 3.3) and establishes the connections between ranks with the transports (Sec. 3.4). When a collective is issued, the host chooses the algorithm and the protocol, splits the work across the channels (Sec. 3.8) and launches a kernel. A helper thread, the *proxy* (Sec. 3.5), serves the transports that cannot be driven by the GPU alone.

Device side. The kernel runs one thread block per channel. The thread block executes the algorithm as a sequence of calls to the `Primitives` abstraction (Sec. 3.9), which moves data between buffers and between ranks using the selected protocol.

The choice of the algorithm (ring, tree, ...) and of the protocol is made by a tuning model that estimates, for each pair, the latency and bandwidth from the topology and the message size, and picks the pair with the lowest expected time. The

kernel for a given collective is selected through a function identifier (`devFuncId`) that encodes the collective, the algorithm, the protocol, the datatype and the reduction operator; the kernels are therefore instantiated at compile time for each combination.

3.2 Topology and graphs

At initialisation NCCL builds a model of the machine: GPUs, CPUs, PCIe switches, NVLink and NVSwitch connections, and network interfaces, each link with a type and a bandwidth. From it, it computes the paths between every pair of devices and then searches, for each family of algorithms, a *graph* (`ncclTopoGraph`) that describes how the ranks should be connected to obtain a high bandwidth on this specific machine. The search produces, for every pattern, a number of *channels* and, for every channel, an ordering of the GPUs inside the node and the network interface used to enter and leave the node.

Two graphs are of interest in this thesis.

- The *ring graph*: each channel is a ring that visits all the ranks; inside a node the order of the GPUs follows the fastest links, and between nodes the ring leaves from and enters into the GPUs closest to a network interface.
- The *tree graph*: inside a node the GPUs are connected in a chain, and the nodes are connected to each other with a double binary tree, in which every node is a leaf in one of the two trees and an internal node in the other, so that the load on the links is balanced (see [4]).

The results of the search are stored in the per-channel structures `ncclRing` and `ncclTree`, which record for each rank its predecessor and successor in the ring, and its parent and children in the tree. These are the peers that the kernels read to know whom to send to and receive from. The Bine tree does not use the tree graph: its peers are computed from the rank numbers alone (Chapter 4), independently of the topology.

3.3 Channels

A *channel* is the unit of parallelism of NCCL. It is an independent set of connections to the peers of the algorithm (the ring neighbours, or the tree parent and children) together with its own buffers, and it is executed by exactly one thread block of the kernel; the kernel is launched with as many blocks as there are channels in use. Different channels move different, disjoint portions of the data concurrently, and may follow different paths in the machine (for example, different rings, or different network interfaces), which is how NCCL uses several links at the same time.

The number of channels is bounded by the graph search and by `MAXCHANNELS`, and for each collective the number actually used depends on the message size, since a small message does not justify the use of many blocks. Every channel has its own copy of the peer information (the `ncclChannel` structure on the host, `ncclDevChannel` on the device), and at the start of the kernel each block copies the state of its channel in the shared memory of the block, in the `ncclShmem` structure. `ncclShmem`

holds the communicator information (rank, number of ranks), the identifier of the channel (`channelId`), the channel structure with the peers of each algorithm, and the description of the work to execute (`ncclDevWorkColl`, Sec. 3.8). Each algorithm adds its own data to the channel structure; the Bine tables of Chapter 4 are stored there, in `channel.bine`.

An important property, used in Chapter 5, is that a connection belongs to a channel: the connections of a channel are created once, at communicator creation, between the peers that the algorithm needs, and the kernel cannot create new ones. An algorithm in which a rank talks to a number of distinct peers that grows with the number of ranks would need as many connections per channel.

3.4 Transports

A *transport* implements the connection between two ranks. For every pair of peers that must communicate on a channel, NCCL selects the best transport that can connect them (the function `canConnect` of each transport), and sets up a *connector* on each side, one for sending and one for receiving. The main transports are:

- `P2P`: between GPUs of the same node connected by NVLink or PCIe. The sender or the receiver can access the memory of the other GPU directly, so the data can be copied with load/store instructions executed by the kernel itself, without the help of the proxy.
- `SHM`: between GPUs of the same node that cannot access each other directly, through a buffer in host shared memory.
- `NET`: between GPUs of different nodes, through a network plugin (InfiniBand verbs, sockets, ...). The kernel only writes the data in a staging buffer and signals the proxy, which posts the network operations.
- Transports for in-network and multicast reduction (`NVLS`, `CollNet`), not used in this thesis.

A connection is a small set of shared structures, mainly the intermediate buffer for each protocol (Sec. 3.6) and a pair of counters. The buffer is divided in `NCCL_STEPS`= 8 *slots* (also called steps). The sender fills a slot and advances the `head`/`tail` counter; the receiver waits on the counter, consumes the slot and advances the other counter, which returns the slot to the sender. This is a producer/consumer FIFO of 8 slots, and it is what allows the sender to proceed up to 8 slots ahead of the receiver without any other synchronisation. The size of a slot (`stepSize`) is the size of the buffer of the protocol divided by `NCCL_STEPS`, and defines the *chunk* (Sec. 3.8.3).

For the transports that allow it, NCCL can also avoid the intermediate buffer and move the data directly between the user buffers of the two ranks (zero-copy). The flags `NCCL_P2P_WRITE` and `NCCL_P2P_READ` in the field `direct` of the work descriptor tell the kernel which of the direct variants of the primitives (Sec. 3.9.2) it may use for the current operation.

3.5 Proxy

GPU kernels cannot, in general, issue network operations. For this reason each process runs a *proxy* service made of host threads: a *progress* thread that executes the network side of the communication, and a service thread used for the connection setup. When a collective that uses the `NET` transport is enqueued, the host also enqueues, for every channel, a *proxy operation* (`ncclProxyOp`) that contains the information that the proxy needs: the channel, the offset and size of the portion of data of that channel (`loopOffset`, `channelSize`), the chunk size and the number of steps. The proxy then drives the network in lockstep with the kernel through the slot counters of Sec. 3.4:

- on the send side, when the kernel has filled a slot in the staging buffer and advanced the counter, the proxy posts the network send for it, and returns the slot to the kernel when the send completes;
- on the receive side, the proxy posts the receive into a free slot, and signals the kernel when the data has arrived.

The kernel therefore sees the network exactly as it sees an intra-node connection, a FIFO of slots; the difference is that on the `NET` transport somebody else (the proxy) moves the slots between the two nodes. Because the proxy has to know in advance the sequence of operations and sizes that the kernel will execute, the proxy operation is derived from the same partition of the work as the kernel (Sec. 3.8.2). This is also a constraint for new algorithms: the sequence of transfers on a connection must be the one that the proxy expects.

3.6 Protocols

A protocol defines how the data and the synchronisation are encoded in the slots of the connection. NCCL has three.

Simple. The data is copied in the slot with no extra encoding. Synchronisation is done separately, with the head/tail counters of the connection, and it requires memory fences to guarantee that the data is visible before the counter is updated. This gives the highest bandwidth but a higher latency, and it is used for large messages.

LL (low latency). Data and a flag are packed in the same 8-byte store (4 bytes of data and a 4-byte flag), so the receiver detects that the data has arrived by polling the flag, with no fences. Latency is low, but half of the bandwidth is used by flags, so it is used for small messages.

LL128. The same idea with 128-byte lines of which 8 bytes are the flag and 120 are data (93.75 % efficiency), relying on the fact that on NVLink-connected GPUs a 128-byte store is delivered atomically and in order. It is used on systems where this is guaranteed.

The protocol determines the size of the buffer, hence of the chunk, and the *grain*, i.e., the granularity at which the data of a channel is split (`ncclProtoGrainSize`: 16 bytes for LL, 512 for Simple, and 1920 bytes for LL128, a whole number of 120-byte data payloads). In the code the protocol is a template parameter of the kernels, and it is chosen at launch together with the algorithm.

3.7 Algorithms

An *algorithm* defines the communication pattern. The ones relevant here are *ring* (each rank communicates only with its predecessor and successor, and data circulates around the ring: $\mathcal{O}(p)$ steps for p ranks), *tree* (the double binary tree of Sec. 3.2, $\mathcal{O}(\log p)$ steps across nodes) and PAT [5] for all-gather and reduce-scatter. Algorithms for in-network reduction (`NVLS`, `CollNet`) exist as well. For every algorithm and collective there is a device function that implements the pattern, and a *pattern* identifier (`ncclPattern_t`) used by the host to compute the number of steps and the chunking. The Bine broadcast and reduce of Chapter 4 are new device functions of this kind, registered as a new algorithm, `NCCL_ALGO_BINE`, with its own pattern identifier (`ncclPatternBine`).

3.8 Work distribution across channels

3.8.1 Traffic-based splitting

When a collective is enqueued, its input is split across the channels so that every channel carries approximately the same amount of *traffic*, and not necessarily the same number of elements. The traffic generated by one byte of user data depends on the collective: for example, an all-reduce moves each byte more than a broadcast does. The host computes for each collective the `trafficPerByte` (increased for the LL protocol to account for the flag overhead), sums the traffic of all the collectives of the plan, and divides it by the number of channels to obtain the `trafficPerChannel`. To avoid splitting small messages among too many channels, the data is not split element by element but in *cells*: a cell is the amount of data that generates a minimum traffic of 32 KiB (`MinTrafficPerChannel`), rounded to a multiple of 16 bytes. A channel receives an integer number of cells.

3.8.2 Continuous byte distribution (CBD)

The partition is called *continuous byte distribution*. The cells are assigned to the channels in order, filling a channel before moving to the next one, so that every channel receives a contiguous range of the buffer. Several collectives issued together (aggregated in the same plan) can be laid out one after the other on the same sequence of channels: a collective may start in the middle of a channel that is partially filled by the previous one. Therefore the channels used by a collective are of three kinds:

- the *lo* channel, `channelLo`, the first one, which only receives the cells that fit in the traffic still available in it (`countLo` elements);

- n_{mid} *mid* channels, which are completely filled and receive the same number of elements (`countMid`);
- the *hi* channel, `channelHi`, the last one, which receives the remainder (`countHi`).

If the collective fits in the lo channel there are neither mid nor hi channels; the remainder of the last channel is also trimmed so that the total is exactly the number of elements of the collective. The channels used are [`channelLo`, `channelHi`], and $n_{mid} =$ `channelHi` $-$ `channelLo` $- 1$. The host writes this in the work descriptor `ncclDevWorkColl`, which is passed to the kernel together with the buffers, the root, the number of warps and the `direct` flags:

```
struct ncclDevWorkColl {
  uint32_t channelLo:8, channelHi:8; // inclusive range
  uint32_t root; void *recvbuff, *sendbuff;
  uint32_t direct:2; // NCCL_P2P_WRITE / NCCL_P2P_READ
  struct { size_t countLo, countMid, countHi;
           uint64_t chunkGrainsLo:21, chunkGrainsMid:21,
                    chunkGrainsHi:21; } cbd;
};
```

Listing 3.1. Fields of `ncclDevWorkColl` that describe the CBD partition (simplified).

`ncclCollCbdPart`. On the device, each block calls

`ncclCollCbdPart`(*work*, *channelId*, *proto*, *eltSize*, *count*, *partOffset*, *partCount*, *chunkCount*)

to obtain its share. Given the channel identifier it returns:

- `partOffset` (called `gridOffset` in the kernels): the offset, in elements, of the slice of the channel; it is 0 for the lo channel, `countLo` $+ m \cdot$ `countMid` for the m-th mid channel (m counted from 0), and `countLo` $+ n_{mid} \cdot$ `countMid` for the hi channel;
- `partCount` (`channelCount`): the number of elements of the slice, i.e., `countLo`, `countMid` or `countHi`;
- `chunkCount`: the number of elements of a chunk (Sec. 3.8.3), equal to the corresponding `chunkGrains` multiplied by the number of elements in a grain of the protocol, `ncclProtoGrainSize`(*proto*)$/$*eltSize*;
- optionally `count`, the total number of elements `countLo` $+ n_{mid} \cdot$ `countMid` $+$ `countHi`, if a non-null pointer is given.

The three sizes are a function of the work descriptor and of the channel identifier only, so every rank, which receives the same descriptor for the same collective, computes the same partition. This is what makes it possible, for the algorithms that need it, to assume that the peer is working on the same region of the buffer as the local rank. The proxy operations of Sec. 3.5 are built from the same values (`loopOffset` and `channelSize` are the slice offset and size in bytes of each channel).

POTREBBE AVER SENSO AGGIUNGERE UNA TABELLA/GLOSSARIO CON LE DEFINIZIONI DI CHANNEL, SLICE, STEP, CHUNK, GRAIN, ...

3.8.3 Chunking

A channel does not move its slice with a single transfer, but a *chunk* at a time. The chunk is bounded by the size of a slot of the connection: for the tree algorithm it is a slot, *stepSize* = *bufferSize*/`NCCL_STEPS`, halved for LL (half of the slot is flags), and scaled by 120/128 for LL128 (the data of a line). The host computes the chunk (`calcCollChunking`; a few algorithms, such as `NVLS` and `CollNet`, shrink it for small messages) and stores it in the descriptor as a number of grains, which `ncclCollCbdPart` converts back to elements. The kernel loops over the chunks of its slice and the last chunk is shorter if the slice is not a multiple of the chunk. Because the slot FIFO has several slots, consecutive chunks are pipelined: while a chunk is being consumed by the receiver the sender can already fill the following ones.

3.9 Device side

3.9.1 Kernel structure

The kernel is launched with one thread block per channel, and the number of threads depends on the protocol and on `nWarps` of the work descriptor. At the start, each block loads in `ncclShmem` the communicator, its channel and the description of the work, and then runs the device function of the collective (for instance, `runRing`, `runTreeUpDown`, or `runBine` for the Bine broadcast and reduce). All the threads of the block execute the device function together, and cooperate in every data movement. The function obtains its slice with `ncclCollCbdPart` (Sec. 3.8.2), and then loops over the chunks, calling the primitives.

3.9.2 Primitives

`Primitives` is the device-side abstraction that hides the transport and the protocol. It is a class template whose parameters are the datatype `T`, the reduction operator `RedOp`, the fan-in/fan-out of the primitive, the use of direct (zero-copy) access, and the protocol `Proto` (Simple, LL or LL128). An object is constructed with the thread identifier and the number of threads, the arrays of the ranks to receive from and to send to, and the input and output buffers. At construction the object loads, from the channel, the connectors towards those peers (buffers and counters), so that the same code works with every transport and protocol.

- `FanSymmetric`$\langle N \rangle$ and `FanAsymmetric`$\langle R, S \rangle$ fix the number of receive peers R and of send peers S at compile time. The ring kernels use `FanSymmetric`$\langle 1 \rangle$ (1 peer in each direction), the tree kernels use asymmetric fans whose receive or send side is the maximum number of children of the tree; a kernel that at every step either receives from one peer or sends to one peer uses $\langle 1, 0 \rangle$ or $\langle 0, 1 \rangle$.

The operations of a `Primitives` object are executed by all the threads of the block on a number of elements n that must fit in a chunk. Offsets are in elements, relative to the input buffer (`inpIx`) or to the output buffer (`outIx`).

`send`(o_{in}, n) reads n elements from the input buffer and sends them to the peer(s).

recv(o_{out}, n) receives n elements from the peer(s) and stores them in the output buffer.

directSend(o_{in}, o_{out}, n) **and directRecv**(o_{out}, n) the same operations on the zero-copy path: the data is exchanged directly with the user buffer of the peer (with a write by the sender, or a read by the receiver) instead of through the slots of the connection; the output offset tells where the data lands in the buffer of the peer. They are usable only when the transport and the registered buffers allow it, as indicated by the `direct` flags of the work descriptor, and otherwise the kernel uses the staged versions.

copySend, directCopySend(o_{in}, o_{out}, n) send n elements and, in addition, store them in the local output buffer.

recvSend, recvCopySend, recvReduceSend, recvReduceCopy, ... fused operations that receive, optionally reduce with the local input, optionally copy to the output, and forward to the next peer in a single pass over the data. They are what makes the ring and the tree efficient, since data is not written to and read back from memory between the receive and the send. The Bine broadcast and reduce of Chapter 4 do not use the ones that also send, since in each step a rank either receives or sends; the reduce uses `recvReduceCopy`, which stops at the reduction and the copy.

sendFromOutput, directSendFromOutput(o, n) send n elements read from the output buffer instead of the input buffer, to forward a partial result that has been accumulated there.

Under the hood the data movement is done by `reduceCopy`, a device routine that reads from a number of sources, optionally reduces them with the operator, and writes to a number of destinations, with a protocol-specific load/store path. It can also be called directly by a kernel, and it is used, with one source and one destination, as a plain copy, for example to copy the local buffer when the input and output buffers are different.

3.9.3 Writing a new kernel

The structure that every kernel in NCCL follows, and that the Bine kernels reproduce, can be summarised as

1. read the descriptor of the work from `ncclShmem`;
2. obtain the slice and the chunk size of the channel with `ncclCollCbdPart`;
3. for each chunk of the slice and each step of the algorithm, determine the peers of the rank in that step, instantiate the `Primitives` with these peers (or reuse one), and call the operation that corresponds to the role of the rank in the step.

The pattern of the communication of each step is therefore the only part that is specific to the algorithm, which is the part that the Bine schedule replaces.

Chapter 4

Bine Tree Algorithms

4.1 Schedule construction

This section describes how the schedule of the Bine tree is computed and used in NCCL. The Bine tree, its negabinary representation and its properties are described in Sec. 2.2.5. The schedule is built for $p = 2^s$ ranks and for root 0, and is defined only for power-of-two numbers of ranks. Any other root is obtained by rotating the root-0 schedule (Sec. 4.1.5).

4.1.1 Output and notation

The schedule is stored in two tables of size $p \times s$,

$$\mathcal{S}(r, i) \in \{0, \dots, p-1\} \cup \{\perp\}, \tag{4.1}$$
$$\mathcal{R}(r, i) \in \{0, \dots, p-1\} \cup \{\perp\}, \tag{4.2}$$

for $r \in [0, p-1]$ and $i \in [0, s-1]$. $\mathcal{S}(r, i) = q$ means that rank r sends to rank q at step i, and $\mathcal{R}(r, i) = q$ means that r receives from q at step i. The value $\perp$ means that r does not communicate at step i. The tables are filled by

```
void ncclGetBineTree(int nRanks, int steps,
                     int *sendTable, int *recvTable);
```

which writes two flat arrays of $p \cdot s$ integers (`sendTable` and `recvTable`), with -1 encoding $\perp$, and asserts that $2^s = p$.

The construction uses the quantities defined in Sec. 2.2.5: the mask M_i (`((1 << (steps - step)) - 1` in the code), the partner $\pi_i(r)$ (Eq. (2.8)), and the run $u(r)$ and receive step i_r (Eq. (2.9)). In the code, $u(r)$ is computed by scanning the bits of $rank2nb(r)$ from the least significant one until the first that differs, capped at s bits, and for the root we set $i_0 = -1$, since it already holds the data before step 0.

4.1.2 Definition of the schedule

A rank receives exactly once, at step i_r, from its parent, and forwards the data at each subsequent step $i > i_r$. Therefore, for every rank r and step i,

$$\mathcal{R}(r,i) = \begin{cases} \pi_i(r) & \text{if } r \neq 0 \text{ and } i = i_r, \\ \perp & \text{otherwise,} \end{cases} \tag{4.3}$$

$$\mathcal{S}(r,i) = \begin{cases} \pi_i(r) & \text{if } i_r < i \leq s-1, \\ \perp & \text{otherwise.} \end{cases} \tag{4.4}$$

The parent of $r \neq 0$ is $\pi_{i_r}(r)$, and the children of r are $\{\pi_i(r) \mid i_r < i \leq s-1\}$. The root has $\overline{0} = 0 \cdots 0$ and $i_0 = -1$, so it sends at every step, to the rank whose negabinary representation is M_i:

$$\mathcal{S}(0,i) \;=\; nb2rank(M_i) \;=\; \Big(\sum_{j=0}^{s-i-1} (-2)^j\Big) \bmod p. \tag{4.5}$$

4.1.3 Example: $p = 8$

For $p = 8$ ($s = 3$) the masks are $M_0 = 111$, $M_1 = 011$, $M_2 = 001$, and the representations $\overline{r}$, the runs $u(r)$ and the receive steps i_r are those of Table 2.4. Rank 3 was worked out in Sec. 2.2.5. The resulting edges for all ranks are $0 \to 3$ at step 0; $0 \to 7$ and $3 \to 4$ at step 1; $0 \to 1$, $3 \to 2$, $7 \to 6$ and $4 \to 5$ at step 2. The tables are ("–" stands for $\perp$):

	$\mathcal{S}(r,i)$			$\mathcal{R}(r,i)$		
r	i=0	i=1	i=2	i=0	i=1	i=2
0	3	7	1	–	–	–
1	–	–	–	–	–	0
2	–	–	–	–	–	3
3	–	4	2	0	–	–
4	–	–	5	–	3	–
5	–	–	–	–	–	4
6	–	–	–	–	–	7
7	–	–	6	–	0	–

4.1.4 Computing the tables

Algorithm 1 computes both tables. For each rank it computes the receive step, registers the edge with the parent, and then registers the edges with the children. Each edge is written on both endpoints at once ($\mathcal{S}$ on the sender, $\mathcal{R}$ on the receiver), so the tables are consistent by construction.

Algorithm 1 Schedule of the Bine tree rooted at rank 0 ($p = 2^s$).

Require: $p = 2^s$
Ensure: tables $\mathcal{S}, \mathcal{R}$ of size $p \times s$
$\mathcal{S}(r,i) \leftarrow \perp,\ \mathcal{R}(r,i) \leftarrow \perp$ for all $r \in [0, p-1]$, $i \in [0, s-1]$
for $r \leftarrow 0$ **to** $p-1$ **do**
  $\overline{r} \leftarrow \mathit{rank2nb}(r)$
  if $r = 0$ **then**
    $i_r \leftarrow -1$ ▷ root: sends at every step
  else
    $u \leftarrow$ number of consecutive LSBs of $\overline{r}$ equal to its LSB
    $i_r \leftarrow s - u$ ▷ Eq. (2.9)
    $\mathit{parent} \leftarrow \mathit{nb2rank}(\overline{r} \oplus M_{i_r})$
    $\mathcal{R}(r, i_r) \leftarrow \mathit{parent}; \quad \mathcal{S}(\mathit{parent}, i_r) \leftarrow r$
  end if
  for $i \leftarrow i_r + 1$ **to** $s-1$ **do**
    $q \leftarrow \mathit{nb2rank}(\overline{r} \oplus M_i)$ ▷ Eq. (2.8)
    $\mathcal{S}(r,i) \leftarrow q; \quad \mathcal{R}(q,i) \leftarrow r$
  end for
end for

The construction costs $O(s)$ operations per rank, plus the conversions, i.e., $O(p \log p)$ in total, and the tables occupy $O(p \log p)$ memory. A rank $r \neq 0$ has $u(r) - 1$ children and the root has s, so the tree has $p - 1$ edges.

4.1.5 Arbitrary root

Only the root-0 tables are generated. For a collective with root $t \in [0, p-1]$, rank r is mapped to the virtual rank $r' = (r - t) \bmod p$, the root-0 table is read at r', and the peer found is mapped back by adding t:

$$\mathcal{S}_t(r,i) = \begin{cases} (\mathcal{S}(r',i) + t) \bmod p & \text{if } \mathcal{S}(r',i) \neq \perp, \\ \perp & \text{otherwise,} \end{cases} \qquad r' = (r - t) \bmod p, \tag{4.6}$$

and likewise for $\mathcal{R}_t$. The rotation is a relabeling of the ranks, so it preserves the structure of the tree and the modular distances between communicating ranks. A single table of $p \cdot s$ entries per direction therefore serves every root.

The lookup is implemented by the inline functions `ncclBineTreeSend` and `ncclBineTreeRecv`, which differ only in the table they receive and both call

```
NCCL_HOST_DEVICE_INLINE int
ncclBineTreeLookup(const int *virtualTable, int nRanks, int steps,
                   int root, int rank, int step)
{
  const int vrank = pmod(rank - root, nRanks);
  const int vpeer = virtualTable[idx(vrank, step, steps)];
  return (vpeer < 0) ? -1 : pmod(vpeer + root, nRanks);
}
```

where `pmod` is the non-negative modulo and `idx(r, i, s)` is the index of entry (r, i) in the flat table. The functions are host and device inline, so they can be called from both host and device code, and cost $O(1)$ per step. The returned value -1 means that the rank does not communicate at that step.

4.2 Broadcast and reduce

The tables describe the communication pattern only; which data moves at each step is decided by the collective. This section uses the Bine tree for broadcast and reduce, where every edge carries the whole data: the broadcast sends the buffer of the root along the tree rooted at `work->root`, and the reduce uses the same edges in the opposite direction and combines the contributions at the root.

4.2.1 Integration in NCCL

NCCL implements only the ring natively for these two collectives, so Bine is added as a new algorithm, `NCCL_ALGO_BINE`, with three pieces.

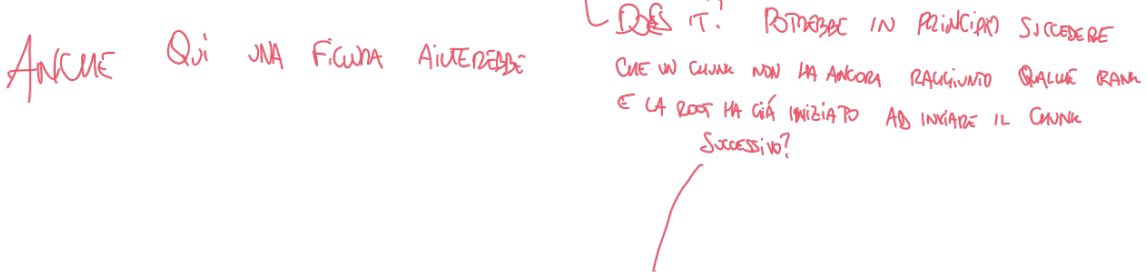
- A pattern, `ncclPatternBine`, which the host uses to compute the number of steps and the chunking.
- The root-0 tables of Sec. 4.1, stored in `channel.bine` and read through the lookup of Sec. 4.1.5.
- One `RunWorkColl` specialization for each collective and protocol, which selects the protocol parameters and calls the common device function `runBine`.

4.2.2 Kernels

Structure of `runBine`. A block runs `runBine` for one channel. It obtains its slice and the chunk size with `ncclCollCbdPart` (Sec. 3.8.2), looks up its peer at each step in the tables rotated by the root (Sec. 4.1.5), and calls the `Primitives` operation that matches its role in that step (Sec. 3.9.2). By Eqs. (4.3) and (4.4), at each step a rank either receives or sends, so the fan is always $\langle 1, 0\rangle$ or $\langle 0, 1\rangle$, and steps in which the rank is idle are skipped. The kernels implement no data movement themselves: all transfers, and in the reduce the combination of the data, are done by the `Primitives`. The two kernels differ in the direction of the walk (root to leaves for the broadcast, leaves to root for the reduce), in the loop order, and in the operations used (Table 4.1). Each operation has a zero-copy variant, which replaces the staged one when `work->direct` allows it; the algorithms below show the staged names only.

Loop order and pipelining. The loop order determines how much of the tree is traversed concurrently.

- *Broadcast (chunk-outer).* For each chunk of the slice, the rank walks the steps from 0 to $s - 1$, so a chunk crosses the whole tree before the same rank starts the next one. Within a chunk, an internal rank is store-and-forward: it first



receives the chunk and then sends it to its children, one per step. Consecutive chunks overlap across levels, since a parent can send chunk $c+1$ while its child is still forwarding chunk c, up to the depth of the slot FIFO (Sec. 3.4).

- *Reduce (step-outer).* For each step, from $s-1$ down to 0, the rank processes its whole slice: it receives and accumulates the slice from one child before moving to the next, and it sends the result to its parent only after the last child. A parent therefore cannot start before the full slice has arrived from the child it is waiting on, and the levels of the tree are executed essentially in series.

Chunking in the Bine kernels thus only hides latency across levels (and, in the broadcast, across consecutive chunks); it does not amortise the bandwidth term as in the pipelined ring of Eq. (2.4). In the broadcast the root still sends every chunk to its s children, and in the reduce it still receives s full slices, so the bandwidth term stays at $s\,n\beta$ as in Eq. (2.5). This is the reason why Bine loses against the ring for large vectors.

Construction of the primitives. A `Primitives` object is bound to its peers at construction, when it loads the connectors towards them, so a new object is needed whenever the peer or the direction changes. In the broadcast it is constructed inside the step loop, hence for every chunk and every step in which the rank is active; in the reduce it is constructed once per step and reused for all the chunks of the slice. Since the fan of an object is fixed at compile time, the kernel instantiates the two fans $\langle 1, 0\rangle$ and $\langle 0, 1\rangle$ and chooses between them with the role of the rank in the step. The cost of this setup, and the fact that no object receives and sends at the same time, are the main differences from the fused kernels of the ring (Sec. 4.4.2).

Zero-copy selection. The broadcast reads the two direct flags separately, `NCCL_P2P_READ` for the receive and `NCCL_P2P_WRITE` for the send, and always instantiates the primitives with `Direct`$= 1$, deciding at run time between the direct and the staged call. The reduce tests the two flags together and, at run time, takes the `Direct`$= 1$ or the `Direct`$= 0$ instantiation for both directions.

Buffers and root output.

- *Broadcast.* The root sends from `sendbuff`, while the other ranks work in place on `recvbuff`. When `sendbuff`$\neq$`recvbuff`, the staged send does not write the output of the root, so after the loop the root copies its slice into `recvbuff` with `reduceCopy`.
- *Reduce.* `recvbuff` is the accumulator: the first receive combines `sendbuff` with the child's data, later ones accumulate in place, and a leaf sends directly from `sendbuff`. The root's last receive writes the result, so no root copy is needed.

Operation counts. A non-root rank does one receive and $u(r)-1$ sends in the broadcast, and $u(r)-1$ receives and one send in the reduce; the root does s sends or s receives. For $p = 8$ and root 0, rank 3 receives from rank 0 at step 0 and sends

to 4 and 2 at steps 1 and 2 in the broadcast; in the reduce it receives from 2 and 4 at steps 2 and 1 and sends to 0 at step 0.

LL and LL128. These protocols have no zero-copy path, so `directRecvReduceCopy` had to be added to `PrimitivesWithoutDirect` in `primitives.h` for the reduce kernel to be instantiated with them.

Algorithm 2 Bine broadcast (`runBine`, one block per channel).

1: $(grid, count, chunk) \leftarrow$ CollCbdPart
2: **for** $e \leftarrow 0$ **to** $count - 1$ **step** $chunk$ **do** ▷ chunks, pipelined across levels
3: $o \leftarrow grid + e$; $n \leftarrow \min(chunk,\ count - e)$
4: **for** $i \leftarrow 0$ **to** $s - 1$ **do** ▷ root to leaves
5: **if** $\mathcal{R}_t(r, i) \neq \perp$ **then**
6: `recv`(o, n) ▷ from the parent, into `recvbuff`
7: **else if** $\mathcal{S}_t(r, i) \neq \perp$ **then**
8: `send`(o, n) ▷ to a child, forwarding from `recvbuff`
9: **end if**
10: **end for**
11: **end for**
12: **if** $r = t \wedge$ `sendbuff` $\neq$ `recvbuff` **then**
13: `reduceCopy`(`sendbuff` $\rightarrow$ `recvbuff`) ▷ the staged `send` does not write the root's output
14: **end if**

Algorithm 3 Bine reduce (`runBine`, one block per channel).

1: *reduced* $\leftarrow$ **false**
2: **for** $i \leftarrow s - 1$ **down to** 0 **do** ▷ leaves to root
3: **if** $\mathcal{S}_t(r, i) \neq \perp$ **then** ▷ tables swapped: the send entry is a child
4: **for** each chunk (o, n) of the slice **do**
5: `recvReduceCopy`(o, n) ▷ accumulates in `recvbuff`
6: **end for**
7: *reduced* $\leftarrow$ **true**
8: **else if** $\mathcal{R}_t(r, i) \neq \perp$ **then** ▷ the receive entry is the parent
9: **for** each chunk (o, n) of the slice **do**
10: **if** *reduced* `sendFromOutput`(o, n) **else** `send`(o, n)
11: **end for**
12: **end if**
13: **end for**

4.3 Testing

The schedule is a deterministic function of $p = 2^s$ and of the root t, and every property the collectives rely on is a statement about finite tables. We therefore test it exhaustively against a set of invariants, for every $p = 2^s$ with $1 \leq s \leq 12$ (up

Table 4.1. NCCL operations used by the Bine broadcast and reduce. The direct operation is used when the zero-copy path is available.

	Role	Fan	Direct	Otherwise
Broadcast	Receive from the parent	$\langle 1, 0\rangle$	`directRecv`	`recv`
	Send, root with `sendbuff`=`recvbuff`	$\langle 0, 1\rangle$	`directSend`	`send`
	Send, any other rank	$\langle 0, 1\rangle$	`directCopySend`	`send`
	Root with `sendbuff`≠`recvbuff`, after the loop	–	`reduceCopy` (copy)	
Reduce	Receive from a child (first or later)	$\langle 1, 0\rangle$	`directRecvReduceCopy`	`recvReduceCopy`
	Send to the parent, leaf	$\langle 0, 1\rangle$	`directSend`	`send`
	Send to the parent, after a receive	$\langle 0, 1\rangle$	`directSendFromOutput`	`sendFromOutput`

to $2^{12} = 4{,}096$ ranks), every root $t \in [0, p-1]$, every rank and every step. The invariants are checked on the rooted schedule $\mathcal{S}_t, \mathcal{R}_t$ obtained through the rotation of Eq. (4.6), not only on the root-0 tables, so that they also validate the rotation.

4.3.1 Invariants

For a root t, let $\rho_t(r)$ be the first step at which rank r receives, with $\rho_t(t) = -1$ for the root and $\rho_t(r) = s$ if a non-root rank never receives. Let $r' = (r - t) \bmod p$ denote the virtual rank of r. For every $p = 2^s$, root t, rank r and step $i \in [0, s-1]$:

(I1) *Single reception.* The number of steps i with $\mathcal{R}_t(r, i) \neq \perp$ is 1 if $r \neq t$ and 0 if $r = t$.

(I2) *Send/receive symmetry.* If $\mathcal{S}_t(a, i) = b \neq \perp$, then $\mathcal{R}_t(b, i) = a$.

(I3) *Causality.* If $\mathcal{S}_t(r, i) \neq \perp$, then $i > \rho_t(r)$. In particular, the root may send from step 0.

(I4) *No self-communication.* $\mathcal{S}_t(r, i) \neq r$ and $\mathcal{R}_t(r, i) \neq r$.

(I5) *Reachability.* Every rank $r \neq t$ has $\rho_t(r) \leq s - 1$.

(I6) *The root never receives.* $\mathcal{R}_t(t, i) = \perp$ for all i.

(I7) *Partner differs in the $s - i$ least significant bits.* If q is the send or the receive peer of r at step i, then

$$\overline{r'} \oplus \overline{q'} = M_i, \qquad q' = (q - t) \bmod p. \tag{4.7}$$

4.3.2 Why they hold and what they guarantee

Tree structure (I1)–(I6). These invariants are structural: they do not depend on how the partners are chosen. Together they say that the edges $\{a \xrightarrow{i} b \mid \mathcal{S}_t(a, i) = b\}$ form a spanning tree rooted at t along which the steps strictly increase. The argument has three parts.

Every rank has exactly one parent. A non-root rank r receives only at the step $i_r = s - u(r)$ of Eq. (2.9), and $u(r)$ is a single number, so r has exactly one incoming edge (I1). For the root, $i_0 = -1$ and no receive is ever defined (I6). Rotation is a

bijection on the ranks and does not change the steps in which a rank communicates, so this holds for every root. That the incoming edge of r is the send of its parent at the same step is the symmetry (I2), argued next.

Data flows forward in time. By Eq. (4.4) a rank sends only at steps $i > i_r$, and from step 0 for the root (I3). Hence the parent a of a rank r satisfies $\rho_t(a) < \rho_t(r)$, so following parent pointers strictly decreases the receive step. The walk cannot enter a cycle and ends at a rank without an incoming edge, which by (I1) and (I6) can only be the root. In particular, the data a rank forwards is always already available.

Every rank is reached, and no edge is degenerate. For every $r \neq 0$ the representation $\overline{r}$ has at least one least significant bit, so $1 \leq u(r) \leq s$ and $i_r \in [0, s-1]$: every non-root rank has a receive step inside the schedule (I5). For $i \leq s-1$ the mask M_i has at least one bit set, so $\overline{r} \oplus M_i \neq \overline{r}$, and since *nb2rank* inverts *rank2nb*, $\pi_i(r) \neq r$ (I4).

Consequently the tree has $p-1$ edges, a broadcast on it completes in s steps with every rank holding the data before it forwards it, and the reversed tree (exchanging the roles of $\mathcal{S}_t$ and $\mathcal{R}_t$) gives the corresponding reduce (Sec. 4.2).

Sender and receiver agree (I2). Both endpoints compute the partner with Eq. (2.8), so $q = \pi_i(r)$ must receive at step i and point back to r. Let $k = s - i$. Rank r sends at step $i > i_r$, i.e., when $k < u(r)$, so the $k+1$ low bits of $\overline{r}$ are equal. The mask flips the k lowest of them and leaves bit k alone, so $\overline{q}$ has a run of exactly k bits: $u(q) = k$ and $i_q = i$. XOR with M_i is an involution, hence $\pi_i(q) = r$. The converse is the same argument read backwards, and the root is the case $\overline{r} = 0$.

Identifying the Bine tree (I7). Invariants (I1)–(I6) are also satisfied by any binomial tree, so they do not identify the Bine tree; (I7) does. It is the defining equation of the partner, Eq. (2.8), rewritten as $\overline{r} \oplus \overline{\pi_i(r)} = M_i$, and it holds for the receive peer as well, since the parent of r is $\pi_{i_r}(r)$ and XOR is symmetric. It is stated on the virtual ranks because the root-t schedule is the root-0 schedule relabeled by the rotation. For given r and i it admits exactly one peer, because XOR with M_i and *nb2rank* are bijections. Hence the tables coincide with those defined by Eq. (2.8), and they inherit the properties of the Bine tree discussed in Sec. 2.2.5, in particular the distance between communicating ranks.

4.3.3 Scope and limits

The tests are exhaustive over all (root, rank) pairs for every power of two $p \leq 2^{12}$, with a cost of $O(p^2 s)$ lookups per value of p. Beyond this range, correctness relies on the arguments of this section, not on testing. The test of (I7) uses the same conversion *rank2nb* as the generator, so it checks that the tables realize the defining equation; the independent check of the construction as a whole comes from the structural invariants (I1)–(I6).

4.4 Experimental results

This section compares the Bine broadcast and reduce of Sec. 4.2 with the native NCCL ring (Chapter 3), under the `SIMPLE` and `LL` protocols.

Setup. Experiments run on the MeluXina supercomputer (see the acknowledgments), on nodes with four NVIDIA A100 GPUs connected by NVLink. We use $p \in \{4, 8, 16, 32, 64, 128\}$ ranks (one to 32 nodes, one rank per GPU), message sizes from 1 kB to 64 MB, and root 0. Each point is the mean over 40 iterations after 10 warmup iterations. Both axes of the figures are logarithmic. Since a node holds four GPUs, $p = 4$ is entirely intra-node, and every larger p also uses the inter-node network. `LL128` is not measured: it relies on 128-byte stores being delivered atomically and in order, it is usually adopted on nv-linked intra-node communication (Sec. 3.6), and NCCL does not enable it on this system, unless forcing it with NCCL_PROTO=LL128, beyond the single-node case $p = 4$, which would not give a comparison across scales.

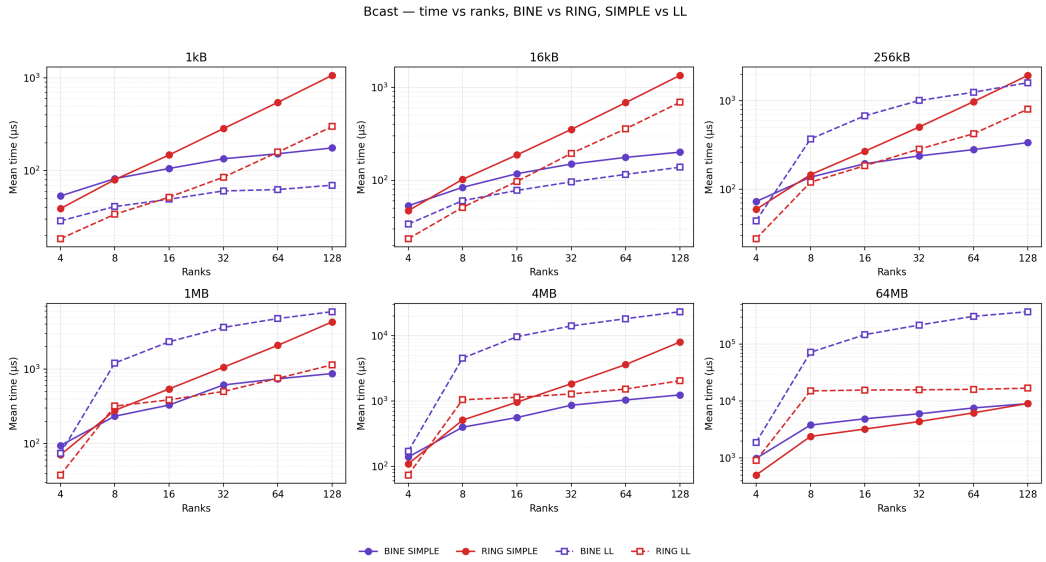


Figure 4.1. Broadcast: mean time versus number of ranks, Bine (blue) against ring (red), for six message sizes. Solid lines with filled markers: `SIMPLE` protocol; dashed lines with hollow markers: `LL` protocol.

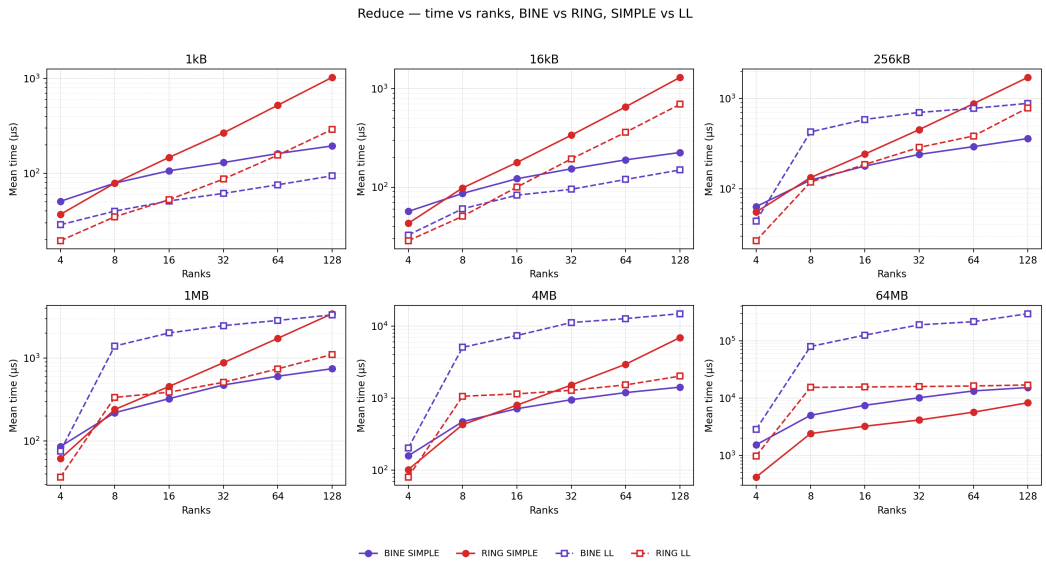


Figure 4.2. Reduce: mean time versus number of ranks, Bine (blue) against ring (red), for six message sizes. Solid lines with filled markers: `SIMPLE` protocol; dashed lines with hollow markers: `LL` protocol.

4.4.1 `SIMPLE` protocol

The solid lines of Figures 4.1 and 4.2 show the expected separation between $\mathcal{O}(p)$ and $\mathcal{O}(\log p)$ (Sec. 2.2.3). Up to 4 MB the ring time grows almost linearly with p (about 2× per doubling), while the Bine time grows by a roughly constant increment per doubling. At $p = 128$ Bine is faster by 5.9×, 6.5×, 5.5×, 4.8× and 6× at 1 kB, 16 kB, 256 kB, 1 MB and 4 MB in the broadcast, and by 5.3×, 5.7× and 4.9× at 1 kB, 16 kB and 256 kB, and by 3.4–3.5× at 1 MB and 4 MB in the reduce. At $p = 4$ the ring is faster for all sizes, and the crossover lies between 4 and 8 ranks: with so few ranks the tree has few steps to save, and $p = 4$ is also the only configuration that stays on NVLink. The measurements do not separate the two effects. At $p = 8$ the two algorithms are within a few percent in the reduce up to 4 MB.

At 64 MB the operation is bandwidth-bound: every edge of the tree carries the whole buffer and the root forwards it s times, whereas the pipelined ring moves each chunk once per link. Bine is therefore slower than the ring in the broadcast for $p \leq 64$ (2× at $p = 4$, 1.2–1.5× for $p = 8$ to 64) and on par at $p = 128$. In the reduce it is slower at every scale (1.8–3.4×), since the root also reduces s full buffers in sequence while the ring spreads the reduction over all ranks. Both algorithms jump sharply between $p = 4$ and $p = 8$, the cost of leaving NVLink for the network.

4.4.2 `LL` protocol

`LL` trades bandwidth for latency (Sec. 3.6): half of the bytes it moves are flags. The dashed lines of Figures 4.1 and 4.2 show that it helps both algorithms at small sizes, but degrades Bine much earlier and much more than the ring.

Table 4.2. Largest measured message size for which `LL` is faster than `SIMPLE`; the crossover lies between this size and the next measured one (1 kB, 16 kB, 256 kB, 1 MB, 4 MB, 64 MB). The Bine and ring entries are the same for broadcast and reduce; the allgather (ring only, Fig. 4.3) was measured up to $p = 32$, a dash means not measured.

	$p = 4$	8	16	32	64	128
Bine (broadcast, reduce)	1 MB	16 kB	16 kB	16 kB	16 kB	16 kB
Ring (broadcast, reduce)	4 MB	256 kB	1 MB	4 MB	4 MB	4 MB
Ring (allgather)	1 MB	16 kB	16 kB	16 kB	–	–

Small messages. Up to 16 kB the picture is that of `SIMPLE` at lower absolute times. At $p = 128$ Bine is 4.4× (broadcast) and 3.1× (reduce) faster than the ring at 1 kB and 4.9× at 16 kB, and Bine itself is 2.6× and 1.4× faster than with `SIMPLE` in the broadcast, at 1 kB and 16 kB respectively. At small p the ring is still better: at $p = 4$ for all sizes, and at $p = 8$ at 16 kB (by 37% in the broadcast and 27% in the reduce).

Large messages. Above 16 kB Bine scales worse than the ring. At 64 MB the ring is almost flat in p (from 15.3 to 17.1 ms in the broadcast between $p = 8$ and 128, about 4 GB/s), while Bine grows from 69.6 to 367.7 ms (from about 1.0 to 0.18 GB/s), becoming 21.5× slower in the broadcast and about 18× in the reduce at $p = 128$. This is expected. `LL` sustains a much lower bandwidth than `SIMPLE`, and the ring is the more bandwidth-efficient algorithm: each byte crosses each link once, while in the tree the root and the internal ranks send the buffer up to s times, so the algorithm that moves more data pays more for the lower bandwidth. For $p \geq 8$ and 64 MB the ring is slowed down by `LL` by 1.8–6.3× with respect to `SIMPLE`, but Bine by 15–41×. In addition, an intermediate rank of the ring uses the fused `directRecvCopyDirectSend` (Sec. 3.9.2): it receives and forwards the same data in a single pass (under `LL`, which has no zero-copy path, this is the staged `recvCopySend`), so every rank receives and sends at the same time and a chunk streams through the ring. In Bine, instead, a rank either receives or sends at each step (fans $\langle 1, 0\rangle$ and $\langle 0, 1\rangle$, Sec. 4.2), with separate operations, so each hop of the tree is store-and-forward at the granularity of a chunk and no rank uses its receive and send directions simultaneously. The slowdown follows the number of steps: at $p = 4$ it is 2.1× in the broadcast, about $s = 2$, and it grows with p.

Crossover between `LL` and `SIMPLE`. As a consequence, `SIMPLE` overtakes `LL` much earlier for Bine than for the ring (Table 4.2): between 16 and 256 kB for every $p \geq 8$ for Bine, against between 256 kB and 1 MB at $p = 8$ and between 4 and 64 MB for $p \geq 32$ for the ring. The two collectives have the same crossover sizes.

Why the crossover differs: the ring allgather as a control. Table 4.2 shows that the ring is not intrinsically tolerant of `LL`: the ring allgather (Fig. 4.3) switches to `SIMPLE` at the same sizes as Bine, at 16 kB for every $p \geq 8$ and at 1 MB for $p = 4$, and the ring with `LL` is slower than with `SIMPLE` from 256 kB on at $p \geq 8$ and from 4 MB on at $p = 4$. Same protocols, same transport and same ring, but a different

crossover from the ring broadcast and reduce. The difference is therefore due to the algorithm, and the latency–bandwidth model of Table 2.2 explains it. The size on the axis of Fig. 4.3 is that of the block contributed by each rank, so $n = p\,m$ in the notation of Table 2.1.

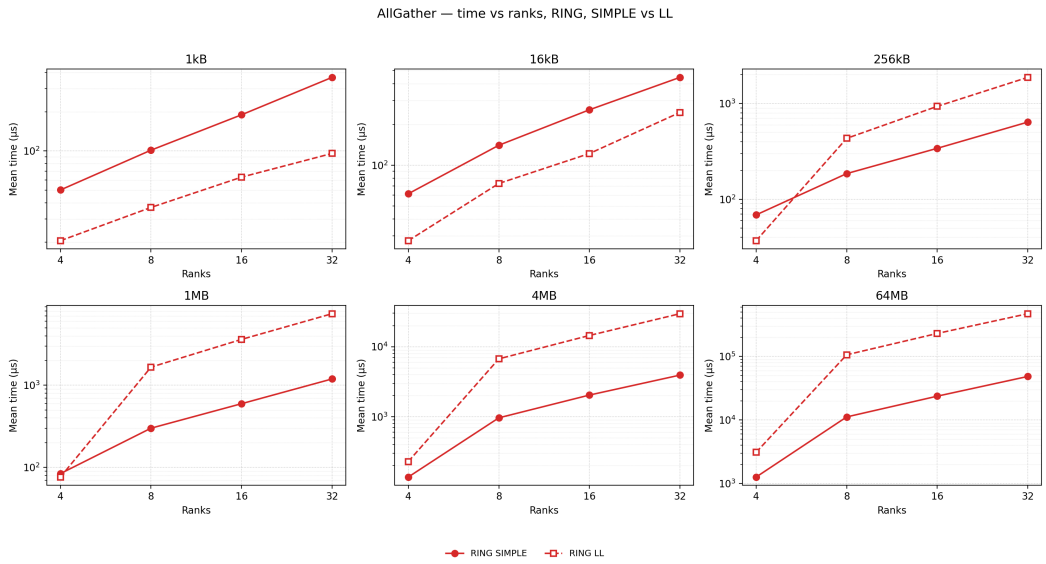


Figure 4.3. Ring allgather: mean time versus number of ranks, `SIMPLE` (solid, filled markers) against `LL` (dashed, hollow markers), for six block sizes, up to $p = 32$.

Let (α_S, β_S) and (α_L, β_L) be the latency and the time per byte of `SIMPLE` and `LL`, with $\alpha_L < \alpha_S$ and $\beta_L > \beta_S$. The ring allgather and the tree broadcast have the same form: a number of steps, each paying α plus the transfer of a whole message,

$$T_{\text{allgather}} = (p-1)\,(\alpha + m\beta), \qquad T_{\text{tree}} = s\,(\alpha + n\beta). \tag{4.8}$$

For such an algorithm `LL` wins if $m < m^* = (\alpha_S - \alpha_L)/(\beta_L - \beta_S)$, and the number of steps cancels: both the latency saved and the bandwidth lost are paid at every step, so the crossover does not depend on p. This is what we observe for the ring allgather and for Bine, whose crossover stays at 16 kB from $p = 8$ to $p = 32$ (to the resolution of our size grid). Expanding Eq. (2.4), the pipelined ring broadcast has instead $T = (p + k - 2)\,\alpha + n\beta + (p-2)\frac{n}{k}\beta$: the bandwidth cost of the vector is paid essentially once, because the chunks are amortized over the steps. Choosing k near k^* keeps the remaining terms small, so to first order the dependence on p is the latency term $(p-2)\alpha$. `LL` then saves about $(p-2)(\alpha_S - \alpha_L)$ and loses about $n(\beta_L - \beta_S)$, so the crossover n^* grows with p, as in the ring row of Table 4.2 (256 kB at $p = 8$, 4 MB from $p = 32$). The same holds for the reduce, which adds $\frac{n}{k}\gamma$ per step and no new term in p. To first order, then, the ring does not tolerate `LL` better than Bine because it is a ring, but because in the broadcast and in the reduce the chunks hide the bandwidth cost, while a collective in which every step moves a full block, like the allgather, does not.

The remaining gap: stepwise primitives. The allgather also gives the reference for the slowdown at 64 MB, where the time is entirely bandwidth. Without

Table 4.3. Best protocol of each algorithm and speedup of Bine over the ring, both with their best protocol, at $p = 128$. A value below 1 means that the ring is faster.

	Broadcast			Reduce		
Size	Bine	Ring	Speedup	Bine	Ring	Speedup
1 kB	LL	LL	4.4×	LL	LL	3.1×
16 kB	LL	LL	4.9×	LL	LL	4.9×
256 kB	SIMPLE	LL	2.3×	SIMPLE	LL	2.2×
1 MB	SIMPLE	LL	1.4×	SIMPLE	LL	1.1×
4 MB	SIMPLE	LL	1.7×	SIMPLE	LL	1.3×
64 MB	SIMPLE	SIMPLE	1.0×	SIMPLE	SIMPLE	0.54×

amortization, but with the fused primitive streaming through the ring, a ring with LL is about 9× slower than with SIMPLE for $p \geq 8$ (and about 2.5× at $p = 4$, which stays on NVLink). Bine is slower than this, 15–41× for $p \geq 8$, and the excess grows with p. This part is not explained by the model of Table 2.2 alone. It is an effect of how the tree kernel is built: at every step and for every chunk a new Primitives object is constructed with fans $\langle 1, 0\rangle$ (receive from the parent) or $\langle 0, 1\rangle$ (send to one child), so the tree is traversed as a sequence of separate point-to-point operations. In the broadcast an internal rank first receives the chunk and then sends it to its children one step at a time; in the reduce it receives from its children one step at a time and sends the result up at the end. Each hop is therefore store-and-forward, a rank never receives and sends at the same time (Sec. 4.4.2, large messages), and the cost of setting up the primitive is paid at every step and chunk. The ring, instead, builds one primitive per kernel and each byte streams through the fused recvCopySend, so the lower bandwidth of LL is paid once. A fused tree kernel would build a single Primitives per rank with FanAsymmetric$\langle 1, D\rangle$ in the broadcast ($\langle D, 1\rangle$ in the reduce), where D is the maximum number of children, receive once and forward to all children in one operation, so that a chunk streams through the tree and the setup is paid once. In summary, the early crossover of Bine is the algorithmic cost of a collective without amortization, shared with the ring allgather, and the additional slowdown at large sizes is due to the stepwise, store-and-forward structure of the kernel.

Effect on the speedup. If each algorithm uses its best protocol (Table 4.3), the advantage of Bine at medium sizes is much smaller than with SIMPLE alone, because the ring can use LL up to 4 MB at $p = 128$ and Bine cannot: at 1–4 MB it drops from 3.4–6× to 1.1–1.7×. At $p = 8$ the ring with LL is faster or on par up to 256 kB, by up to 37% in the broadcast and 27% in the reduce, and Bine keeps a small advantage only at 1 MB and, in the broadcast, at 4 MB. At 64 MB the best protocol is SIMPLE for both, so nothing changes.

4.4.3 Summary

With SIMPLE, Bine is the better choice for small and medium messages (up to a few MB) from 8 ranks on, with speedups of 3.4–6.5× at 128 ranks and logarithmic scaling, while for 64 MB the ring is better or equal, the reduce being the worse

case; at $p = 4$ the ring always wins, but this configuration is entirely intra-node and not representative. `LL` helps Bine only up to 16 kB, where it is up to 4.9× faster than the ring at 128 ranks. Above that the ring, being more bandwidth-efficient, keeps benefiting from `LL` up to 4 MB while Bine must switch to `SIMPLE` (the ring allgather, which has no chunk amortization, switches as early as Bine, so the early crossover is algorithmic, and the extra slowdown of Bine at large sizes is the fan-out and fan-in of the tree), so with each algorithm at its best protocol the advantage at 1–4 MB falls to 1.1–1.7×, and at $p = 8$ it vanishes below 1 MB. This motivates the bandwidth-optimal butterfly-style algorithms of Chapter 5, and a selection rule based on message size, number of ranks and protocol, not on the algorithm alone.

DA QUALCHE PARTE ACCENNI/MOSTRI ANCHE
I TOOL CHE HA SVILUPPATO
(QUELLO PER VISUALIZZARE LE LATENZE SUGLI "ARCHI"
E NCCL-UNIT)

Chapter 5

Butterfly-style Algorithms

[TODO: Butterfly]

Bibliography

[1] Henri Casanova, Arnaud Legrand, and Yves Robert. *Parallel Algorithms*. Sec. 3.3.1, broadcast on a ring. Chapman & Hall/CRC, 2008.

[2] Daniele De Sensi, Saverio Pasqualoni, Lorenzo Piarulli, Tommaso Bonato, Seydou Ba, Matteo Turisini, Jens Domke, and Torsten Hoefler. "Bine Trees: Enhancing Collective Operations by Optimizing Communication Locality". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '25. St. Louis, MO, USA: ACM, 2025. DOI: 10.1145/3712285.3759835. arXiv: 2508.17311. URL: https://arxiv.org/abs/2508.17311.

[3] Roger W. Hockney. "The communication challenge for MPP: Intel Paragon and Meiko CS-2". In: *Parallel Computing* 20.3 (1994), pp. 389–398. DOI: 10.1016/S0167-8191(06)80021-9.

[4] Zhiyi Hu, Siyuan Shen, Tommaso Bonato, Sylvain Jeaugey, Cedell Alexander, Eric Spada, James Dinan, Jeff Hammond, and Torsten Hoefler. *Demystifying NCCL: An In-depth Analysis of GPU Communication Protocols and Algorithms*. 2025. DOI: 10.48550/arXiv.2507.04786. arXiv: 2507.04786 [cs.DC].

[5] Sylvain Jeaugey. *PAT: a new algorithm for all-gather and reduce-scatter operations at scale*. 2025. DOI: 10.48550/arXiv.2506.20252. arXiv: 2506.20252 [cs.DC].

[6] NVIDIA. *NCCL*. URL: https://github.com/NVIDIA/nccl (visited on 10/16/2026).

[7] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. "Optimization of Collective Communication Operations in MPICH". In: *The International Journal of High Performance Computing Applications* 19.1 (2005), pp. 49–66.

[8] Jesper Larsson Träff and Andreas Ripke. "Optimal Broadcast for Fully Connected Networks". In: *High Performance Computing and Communications (HPCC)*. Vol. 3726. LNCS. 2005. DOI: 10.1007/11557654_8.